AHM's

ANJUMAN INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(Affiliated to Visvesvaraya Technological University, Belagavi
and pRecognized by AICTE, New Delhi)
Bhatkal – 581320, Karnataka, India

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

MACHINE LEARNING LABORATORY

BCSL606

SEMESTER: VI

SCHEME-2022

Name of the Student: …….

USN of the Student: ……..

Table of Contents

Sl. No	Title	Page. No
1.	Course Outcomes and Program Outcomes (POs)	**3-4**
2.	General Lab Guidelines	**5**
3.	Syllabus	**6**
4.	Assessment Details- CIE and SEE	**7-8**
5.	Rubrics	**9**
6.	Maintaining Record and Execution Steps	**10**
7.	List of Experiments/Programs with Output	**11-33**

COURSE OUTCOMES

COURSE OUTCOMES	
At the end of the course the student will be able to:	
CO1	Illustrate the principles of multivariate data and apply dimensionality reduction techniques.
CO2	Demonstrate similarity-based learning methods and perform regression analysis.
CO3	Develop decision trees for classification and regression problems, and Bayesian models for probabilistic learning.
CO4	Implement the clustering algorithms to share computing resources.

COURSE LEARNING OBJECTIVES

- To become familiar with data and visualize univariate, bivariate, and multivariate data using statistical techniques and dimensionality reduction. Non-Linear data structures and their applications such as trees and graphs.
- To understand various machine learning algorithms such as similarity-based learning, regression, decision trees, and clustering.
- To familiarize with learning theories, probability-based models and developing the skills required for decision making in dynamic environments.

CO-PO Mapping

CO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11	PO12	PSO1	PSO2	PSO3
1	3	2	2	1	2	-	-	-	-	-	-	3			
2	3	3	3	1	2	-	-	-	-	-	-	3			
3	3	3	3	1	2	-	-	-	-	-	-	3			
4	3	3	3	2	2	-	-	-	-	-	-	3			

PROGRAM OUTCOMES (POs)

PROGRAM OUTCOMES/ GRADUAL ATTRIBUTES	
PO1	**Engineering Knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
PO2	**Problem Analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
PO3	**Design/Development of Solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
PO4	**Conduct Investigations of Complex Problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
PO5	**Modern Tool Usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.
PO6	**The Engineer and Society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
PO7	**Environment and Sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
PO8	**Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
PO9	**Individual and Team work:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.
PO10	**Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
PO11	**Project Management and Finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
PO12	**Life-Long Learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

General Lab Guidelines

Do's

1. Maintain discipline in the laboratory.
2. Before entering the laboratory, keep your bags on the rack placed outside the lab.
3. Proper dress code with ID Cards is mandatory to get entry into the laboratory.
4. Students should carry a lab observation book, manual and record book must be up to date with the latest experiment written neatly.
5. Entry in the lab Utilization Register must be made before switching on the computer.
6. Maintain the same system throughout the course of the subject.
7. Program output with correct logic and program must be noted down in the lab observation book.
8. After completing the experiment, students must switch off the virtual machine first, followed by the main system, arrange the chair and leave the lab after getting the signature from the concerned faculty.

Don'ts

1. Do not come late to the laboratory.
2. Do not leave the laboratory without the permission of the faculty in-charge.
3. Never eat or drink while in the laboratory.
4. Do not alter computer/configuration settings.
5. Do not mishandle computers in the Laboratory.

SYLLABUS

Sl.NO	Experiments
1	Develop a program to create histograms for all numerical features and analyze the distribution of each feature. Generate box plots for all numerical features and identify any outliers. Use California Housing dataset. **Book 1: Chapter 2**
2	Develop a program to Compute the correlation matrix to understand the relationships between pairs of features. Visualize the correlation matrix using a heatmap to know which variables have strong positive/negative correlations. Create a pair plot to visualize pairwise relationships between features. Use California Housing dataset. **Book 1: Chapter 2**
3	Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2. **Book 1: Chapter 2**
4	For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples. **Book 1: Chapter 3**
5	Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of x in the range of [0,1]. Perform the following based on dataset generated. a. Label the first 50 points $\{x_{1,\ldots\ldots},x_{50}\}$ as follows: if ($x_i \leq 0.5$), then $x_i \in Class_1$, else $xi \in Class_2$ b. Classify the remaining points, $x_{51,\ldots\ldots},x_{100}$ using KNN. Perform this for k=1,2,3,4,5,20,30 **Book 2: Chapter 2**
6	Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs. **Book 1: Chapter – 4**
7	Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset for Linear Regression and Auto MPG Dataset (for vehicle fuel efficiency prediction) for Polynomial Regression. **Book 1: Chapter – 5**
8	Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample. **Book 2: Chapter – 3**
9	Develop a program to implement the Naive Bayesian classifier considering Olivetti Face Data set for training. Compute the accuracy of the classifier, considering a few test data sets. **Book 2: Chapter – 4**
10	Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result. **Book 2: Chapter – 4**

ASSESSMENT DETAILS (BOTH CIE AND SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks).

A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each subject/ course if the student secures not less than 35%(18 Marks out of 50) in the semester-end examination (SEE), and a minimum of 40% (40 marks out of 100) in the sum total of the CIE (Continuous Internal Evaluation) and SEE (Semester End Examination) taken together.

CONTINUOUS INTERNAL EVALUATION (CIE):

CIE marks for the practical course are **50 Marks**.

The split-up of CIE marks for record/ journal and test are in the ratio **60:40**

- Each experiment is to be evaluated for conduction with an observation sheet and record write-up. Rubrics for the evaluation of the journal/write-up for hardware/software experiments are designed by the faculty who is handling the laboratory session and are made known to students at the beginning of the practical session.
- Record should contain all the specified experiments in the syllabus and each experiment write up will be evaluated for 10 marks.
- Total marks scored by the students are scaled down to **30 marks** (60% of maximum marks).
- Weightage to be given for neatness and submission of record/write-up on time.
- Department shall conduct a test of 100 marks after the completion of all the experiments listed in the syllabus.
- In a test, test write-up, conduction of experiment, acceptable result, and procedural knowledge will carry a weightage of 60% and the rest 40% for viva-voce.
- The suitable rubrics can be designed to evaluate each student's performance and learning ability.
- The marks scored shall be scaled down to **20 marks** (40% of the maximum marks).
- The Sum of scaled-down marks scored in the report write-up/journal and marks of a test is the total CIE marks scored by the student.

SEMESTER END EVALUATION (SEE):

- SEE marks for the practical course are 50 Marks.
- SEE shall be conducted jointly by the two examiners of the same institute; examiners are appointed by the Head of the Institute.
- The examination schedule and names of examiners are informed to the university before the conduction of the examination. These practical examinations are to be conducted between the schedule mentioned in the academic calendar of the University.
- All laboratory experiments are to be included for practical examination.
- (Rubrics) Breakup of marks and the instructions printed on the cover page of the answer script to be strictly adhered to by the examiners. **OR** based on the course requirement evaluation rubrics shall be decided jointly by examiners.
- Students can pick one question (experiment) from the questions lot prepared by the examiners jointly.
- Evaluation of test write-up/ conduction procedure and result/viva will be conducted jointly by examiners.

General rubrics suggested for SEE are mentioned here, writeup-20%, Conduction procedure and result in -60%, Viva-voce 20% of maximum marks. SEE for practical shall be evaluated for 100 marks and scored marks shall be scaled down to 50 marks (however, based on course type, rubrics shall be decided by the examiners) Change of experiment is allowed only once and 15% of Marks allotted to the procedure part are to be made zero. The minimum duration of SEE is 02 hours

RUBRICS

Split-up of Marks	Practical Sessions	Marks (30)
Continuous Evaluation		
#R1	Observation/Write-up/Execution	15
#R2	Record-Writing	10
#R3	Viva-Voce	05
	Total Marks	30
Internal Assessment Test (IA)		
#R1	Write-up of Procedure/Program	10
#R2	Execution	30
#R3	Viva-Voce	10
	Total Marks	50
		Reduced to 20

Total marks for CIE= 30 + 20= 50 Marks

RULES FOR MAINTAINING LABORATORY RECORD

1. Write your name, USN, Branch and Semester with the correct course name and course code on the outside front cover of the record. Write that same information on the first page inside.
2. Update Table of Contents every time you start each new experiment or topic
3. Always use black pen and write neatly and clearly.
4. Date to be written on the right-side page on the top right side corner.
5. On right-side page

   →Title of experiment

   →Aim/Objectives

   →Program
6. On left side page

   →Algorithm (if necessary)

   →Results

Execution Steps

Steps

1. **Install Python**
   - Download from *www.python.org*
   - Select **"Add Python to PATH"** during installation

2. **Install Jupyter Notebook**

   pip install notebook

3. **Launch Jupyter Notebook**

   jupyter notebook

   Browser opens with Jupyter dashboard
4. **Create New Notebook**
   - Click **New → Python 3**
   - Rename the notebook
5. **Run Program**
   - Write code in a cell
   - Press **Shift + Enter** to execute

Sl. No	Name of the Experiment	Page No.
1.	Data Visualization & Analysis	12-14
2.	Correlation Analysis and Heatmap.	15-16
3.	Principal Component Analysis	17-18
4.	Find-S Algorithm	19
5.	K-Nearest Neighbour algorithm	20-21
6.	Locally Weighted Regression algorithm	22-23
7.	Linear Regression and Polynomial Regression	24-27
8.	Decision tree algorithm	28-30
9.	Naive Bayesian classifier	31-32
10.	K-means clustering	33-24

DATASETS:

California Housing Dataset	https://drive.google.com/file/d/1QUdwg-56Wf3j79xNFCvikMMZk1BVbpfA/view?usp=sharing
Iris Dataset	https://drive.google.com/file/d/1rN9r9ufdaRlfPORgjcJXyGQ1a0ZeS5DF/view?usp=sharing
Find-S Dataset	https://drive.google.com/file/d/1EjxOmKQr947DEsSuTgihS5PR4Gc07zCQ/view?usp=sharing
Boston Housing Dataset	https://drive.google.com/file/d/1-5kKTGIrlLSE5QeFhfkeLRNzbbxmQfmS/view?usp=sharing
Auto MPG Dataset	https://drive.google.com/file/d/1NhxZtIAM9oXXN4X6r0tFjuHYj3RvXJb9/view?usp=sharing
Breast Cancer Dataset	https://drive.google.com/file/d/1ubrRz2HEHU5apmcgWCUcvlGLSvO0J_SJ/view?usp=sharing
Olivetti Faces Dataset	https://drive.google.com/file/d/1MQsZATElVnf2Uiyb3lzFo3VRmQ01RI92/view?usp=sharing

Program 1: Data Visualization & Analysis

Develop a program to create histograms for all numerical features and analyze the distribution of each feature. Generate box plots for all numerical features and identify any outliers. Use California Housing dataset.
California Housing Dataset:

Aim:
To analyze data distribution and detect outliers using histograms, box plots and IQR.

Theory:
Exploratory Data Analysis (EDA) is a crucial step in machine learning that helps understand the structure of data before model building. Visualization techniques such as histograms and box plots help identify skewness, spread, and outliers. Detecting outliers improves data quality and model accuracy.

Program

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

#Read the Dataset as a DataFrame
data= pd.read_csv('california_housing.csv')
data.head()

data.hist(bins=30, figsize=(15, 10))
plt.suptitle('Histograms of Numerical Features')
plt.show()

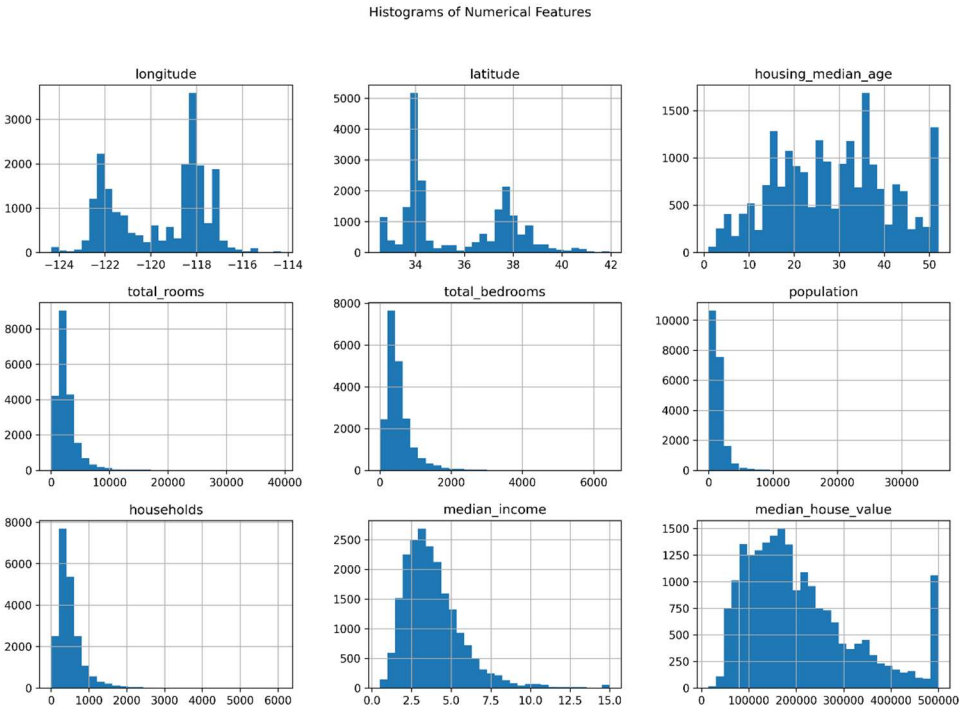
plt.figure(figsize=(15, 10))
for i, column in enumerate(data.select_dtypes(include='num-
ber').columns):
    plt.subplot(3, 3, i+1)
    plt.boxplot(data[column])
    plt.title(column)
plt.suptitle('Box Plots of Numerical Features')
plt.show()

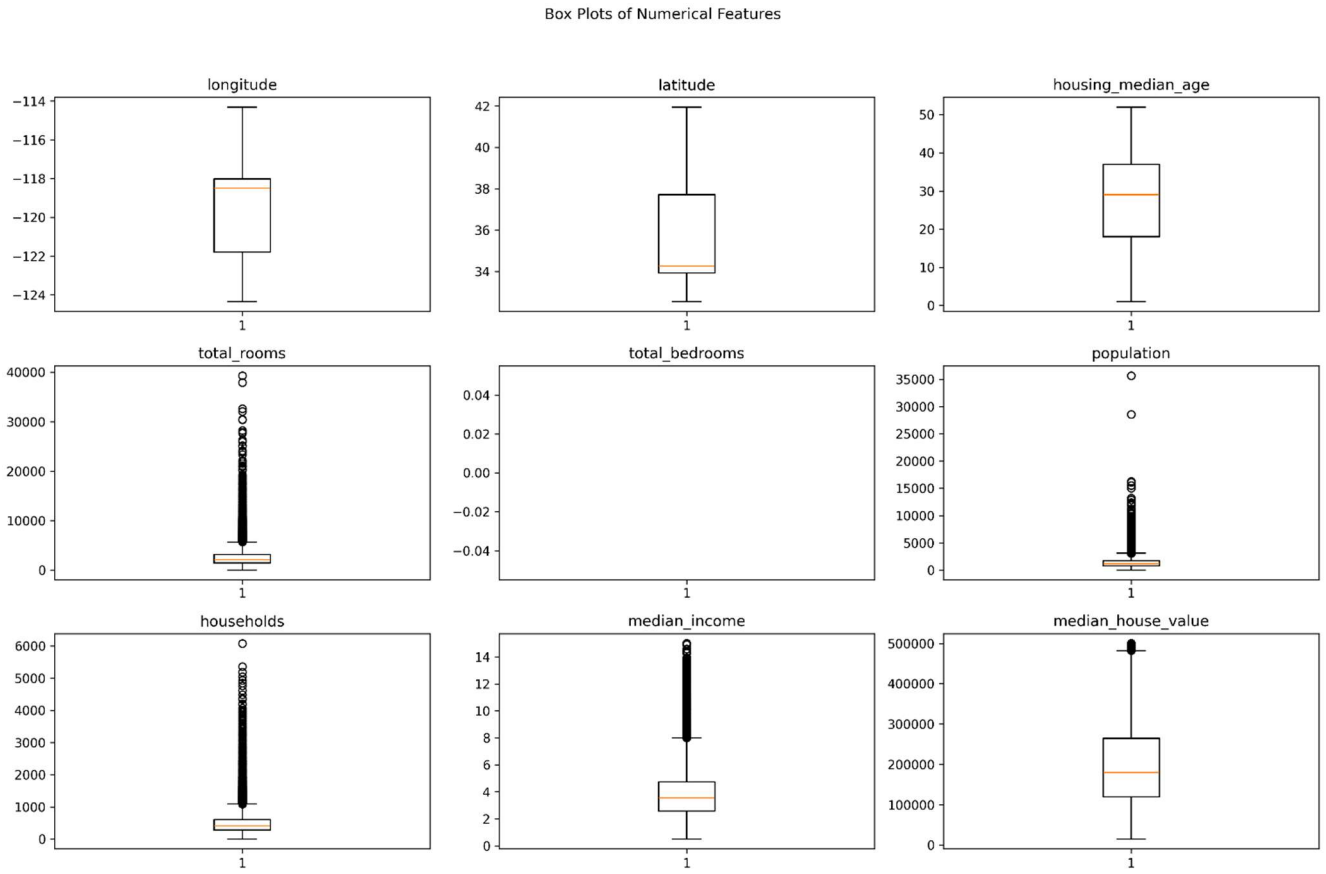
#Identify outliers using the IQR method
Q1 = data.quantile(0.25)  # First quartile (25%)
Q3 = data.quantile(0.75)  # Third quartile (75%)
IQR = Q3 - Q1  # Interquartile range

# Detect outliers
outliers = ((data < (Q1 - 1.5 * IQR)) | (data > (Q3 + 1.5 *
IQR))).sum()

# Print number of outliers per column
print(outliers)
```

Output:


Histograms of Numerical Features
longitude
latitude
housing_median_age
total_rooms
total_bedrooms
population
households
median_income
median_house_value




```
longitude             0
latitude              0
housing_median_age    0
total_rooms        1287
total_bedrooms     1271
population         1196
households         1220
median_income       681
median_house_value 1071
dtype: int64
```

Program 2. Correlation Analysis and Heatmap.

Develop a program to Compute the correlation matrix to understand the relationships between pairs of features. Visualize the correlation matrix using a heatmap to know which variables have strong positive/negative correlations. Create a pair plot to visualize pairwise relationships between features. Use California Housing dataset.

Aim:
To analyze feature relationships using correlation matrix and heatmap.
Theory:
Correlation measures the linear relationship between variables. A heatmap visually represents correlation values and helps identify strongly related features, which is useful for feature selection and multicollinearity detection.
Program:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

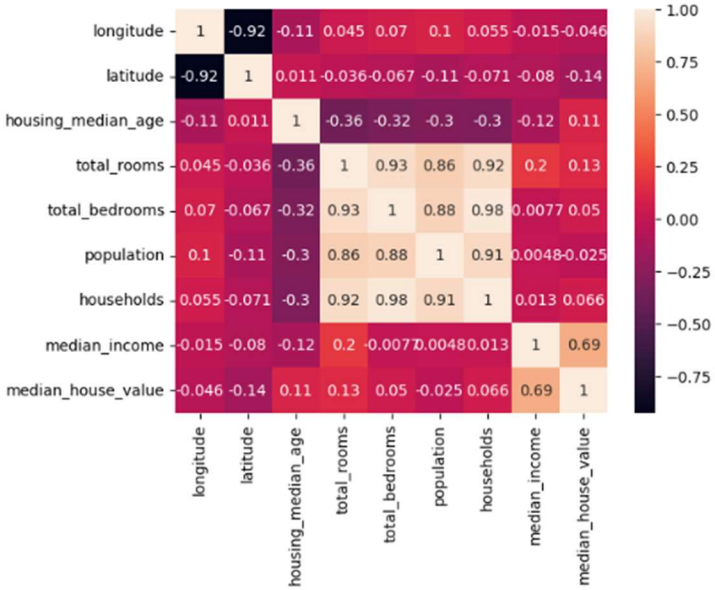
data= pd.read_csv('california_housing.csv')

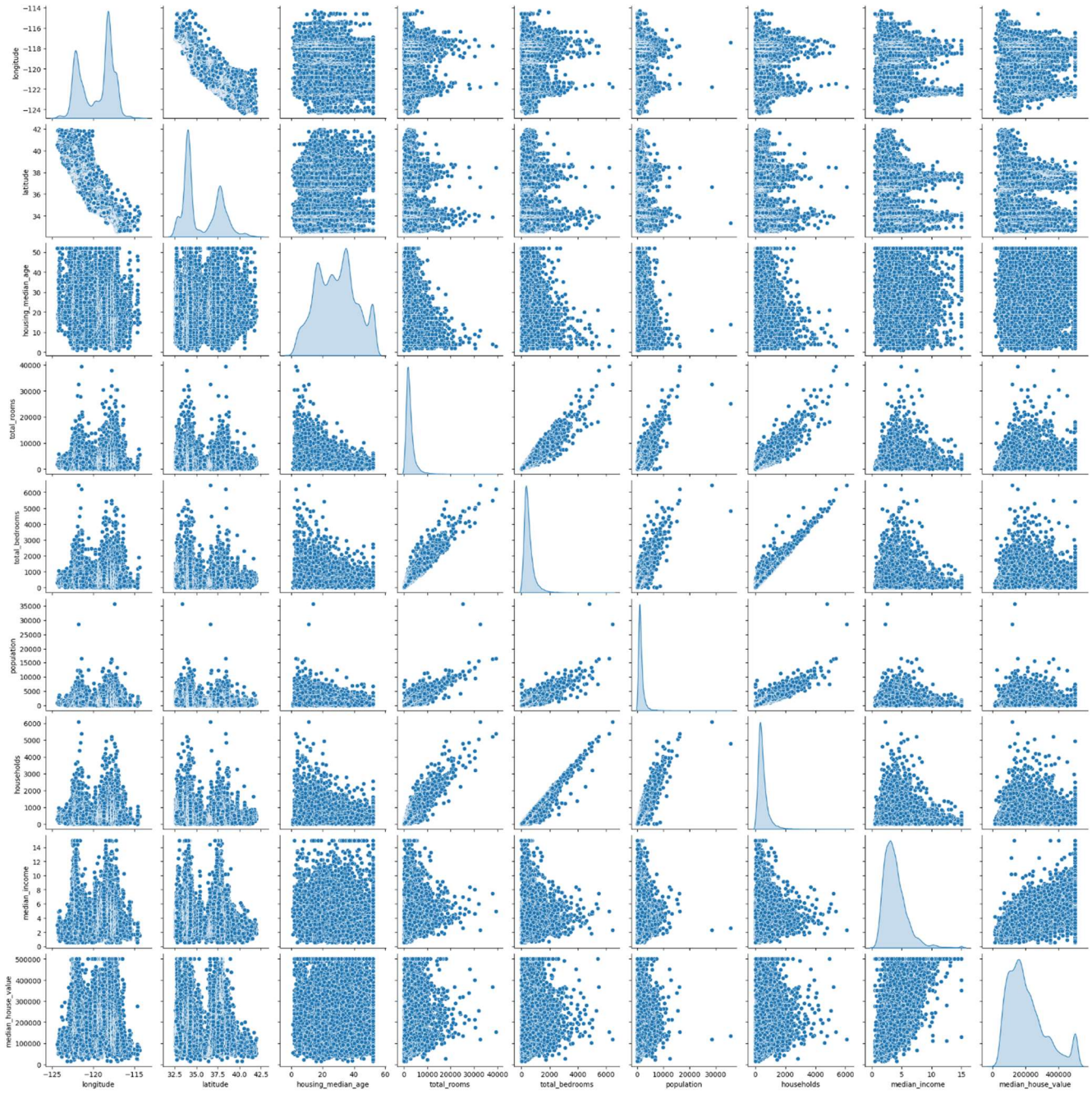
cor_mat= data.select_dtypes(include=['num-
ber']).corr(method='pearson')
print(cor_mat)

sns.heatmap(cor_mat, annot=True)
plt.show()

sns.pairplot(data, diag_kind='kde')
```

Output:




longitude
latitude
housing_median_age
total_rooms
total_bedrooms
population
households
median_income
median_house_value

Program 3: Principal Component Analysis

Develop a program to implement Principal Component Analysis (PCA) for reducing the dimensionality of the Iris dataset from 4 features to 2.

Aim:
To reduce dimensionality of the Iris dataset from 4 features to 2 using PCA.
Theory:
Principal Component Analysis (PCA) is an unsupervised learning technique used for dimensionality reduction. It transforms data into a new coordinate system such that maximum variance is retained in fewer dimensions.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.decomposition import PCA
from sklearn.preprocessing import StandardScaler

data = pd.read_csv("iris.csv")

X = data.iloc[:, 1:5]       # skip Id column
y = data.iloc[:, 5]        # Species column

X = StandardScaler().fit_transform(X)

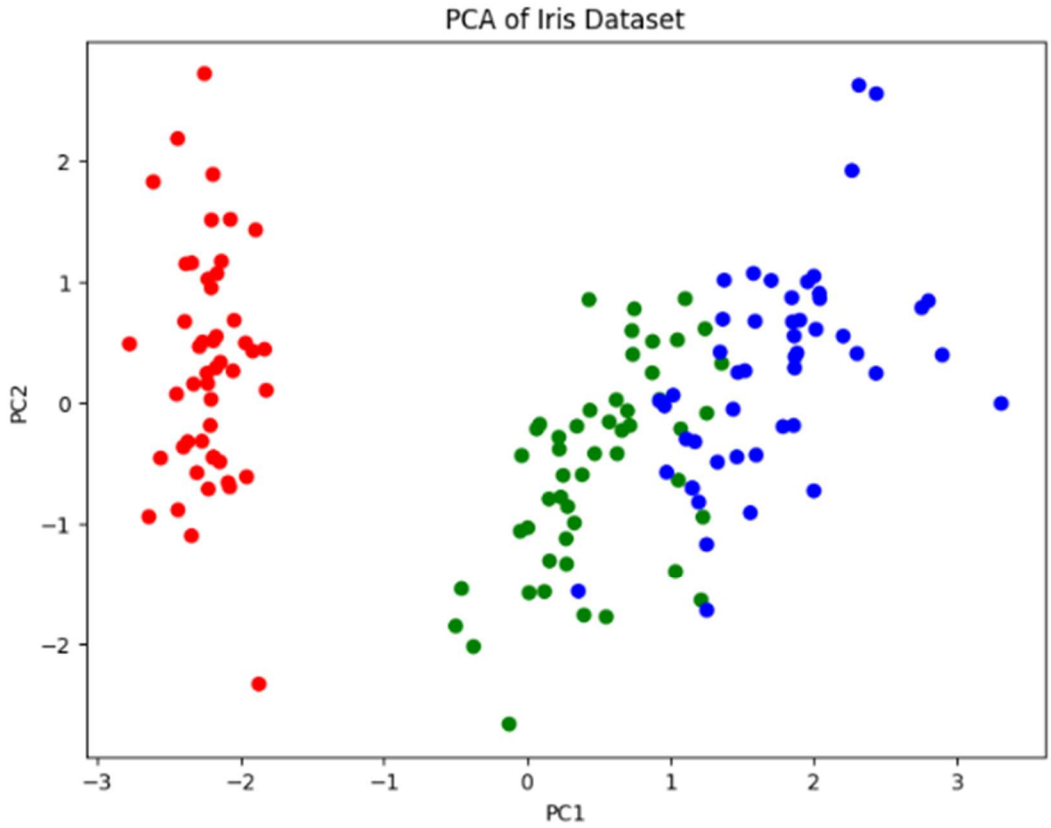
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)

pca_df = pd.DataFrame(X_pca, columns=['PC1', 'PC2'])
pca_df['species'] = y

colors = {
    'Iris-setosa': 'red',
    'Iris-versicolor': 'green',
    'Iris-virginica': 'blue'
}

plt.figure(figsize=(8,6))
plt.scatter(
    pca_df['PC1'],
    pca_df['PC2'],
    c=pca_df['species'].map(colors)
)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA of Iris Dataset")
plt.show()
```

Output:



Program 4: Find-S Algorithm.

For a given set of training data examples stored in a .CSV file, implement and demonstrate the Find-S algorithm to output a description of the set of all hypotheses consistent with the training examples.

Aim:
To obtain the most specific hypothesis using the Find-S algorithm.
Theory:
Find-S is a concept learning algorithm that finds the most specific hypothesis consistent with all positive training examples. It ignores negative examples.
Algorithm:
1. Initialize hypothesis with first positive example.
2. Compare with remaining positive examples.
3. Generalize mismatched attributes.
4. Output final hypothesis.

Program:

```
import pandas as pd
data=pd.read_csv('sport_preferences.csv')
attributes=data.iloc[:, 1:-1].values  # rows, columns: All rows but
Skips first and last column
target=data.iloc[:,-1].values # Only last column , all rows
hypothesis=None

def find_s(data):

#Update hypothesis based on other positive examples
    for i in range(len(target)):
        if target[i] == "Yes":
            hypothesis=attributes[i].copy()
            break
    for i in range(len(target)):
        if target[i] == "Yes":
            for j in range(len(hypothesis)):
                if hypothesis[j] != attributes[i][j]:
                    hypothesis[j] = '?'
            print(hypothesis)
    return hypothesis
final_hypothesis = find_s(data)
# Print the learned hypothesis
print("Most Specific Hypothesis:", final_hypothesis)
```

Output:

```
['Sunny' 'Warm' 'Normal' 'Strong' 'Warm' 'Same']
['Sunny' 'Warm' '?' 'Strong' 'Warm' 'Same']
['Sunny' 'Warm' '?' 'Strong' '?' '?']
Most Specific Hypothesis: ['Sunny' 'Warm' '?' 'Strong' '?' '?']
```

Program 5: k-Nearest Neighbor Algorithm.

Develop a program to implement k-Nearest Neighbour algorithm to classify the randomly generated 100 values of x in the range of [0,1]. Perform the following based on dataset generated.

a. Label the first 50 points $\{x_{1,\ldots\ldots},x_{50}\}$ as follows: if ($x_i \leq 0.5$), then $x_i \in Class_1$, else $xi \in Class_2$
b. Classify the remaining points, $x_{51,\ldots\ldots},x_{100}$ using KNN. Perform this for k=1,2,3,4,5,20,30

Aim:
To classify data using the k-Nearest Neighbour algorithm.
Theory:
k-NN is a supervised learning algorithm that classifies data based on the majority class of its nearest neighbors using distance metrics such as Euclidean distance.
Program:

```
import numpy as np
import pandas as pd
from sklearn.metrics import accuracy_score

# Step 1: Generate 100 random values in [0,1]
data = np.random.rand(100)

# Step 2: Training data (first 50)
X_train = data[:50]
y_train = np.array([1 if x <= 0.5 else 2 for x in X_train])

# Step 3: Test data (remaining 50)
X_test = data[50:]
y_test = np.array([1 if x <= 0.5 else 2 for x in X_test])

# Step 4: KNN function (simplified majority voting)
def knn_predict(X_train, y_train, x_test, k):
    distances = np.abs(X_train - x_test)
    nn_indices = distances.argsort()[:k]
    nn_labels = y_train[nn_indices]
    return int(np.round(np.mean(nn_labels)))

# Step 5: k values
k_values = [1, 2, 3, 4, 5, 20, 30]

# Step 6: Perform classification
for k in k_values:
    y_pred = [knn_predict(X_train, y_train, x, k) for x in
X_test]
    acc = accuracy_score(y_test, y_pred)
    print("\n============================")
    print("k =", k)
    print("Accuracy =",acc)

# Optional
```

```
    df = pd.DataFrame({
        "X_value": X_test,
        "True_Label": y_test,
        "Predicted_Label": y_pred
    })

    print(df.head())    # show first 5 rows only
```

Output:

```
==============================
k = 1
Accuracy = 1.0
    X_value  True_Label  Predicted_Label
0  0.044737           1                1
1  0.836947           2                2
2  0.028190           1                1
3  0.233196           1                1
4  0.682547           2                2

==============================
k = 2
Accuracy = 0.98
    X_value  True_Label  Predicted_Label
0  0.044737           1                1
1  0.836947           2                2
2  0.028190           1                1
3  0.233196           1                1
4  0.682547           2                2

==============================

k = 3
Accuracy = 0.98
    X_value  True_Label  Predicted_Label
0  0.044737           1                1
1  0.836947           2                2
2  0.028190           1                1
3  0.233196           1                1
4  0.682547           2                2

==============================
k = 4
Accuracy = 0.98
    X_value  True_Label  Predicted_Label
0  0.044737           1                1
1  0.836947           2                2
2  0.028190           1                1
3  0.233196           1                1
4  0.682547           2                2

==============================
k = 5
Accuracy = 0.98
    X_value  True_Label  Predicted_Label
0  0.044737           1                1
1  0.836947           2                2
2  0.028190           1                1
3  0.233196           1                1
4  0.682547           2                2
```

```
------------------------------
k = 20
Accuracy = 1.0
    X_value  True_Label  Predicted_Label
0  0.044737           1                1
1  0.836947           2                2
2  0.028190           1                1
3  0.233196           1                1
4  0.682547           2                2

==============================
k = 30
Accuracy = 1.0
    X_value  True_Label  Predicted_Label
0  0.044737           1                1
1  0.836947           2                2
2  0.028190           1                1
3  0.233196           1                1
4  0.682547           2                2
```

Program 6: Locally Weighted Regression Algorithm.

Implement the non-parametric Locally Weighted Regression algorithm in order to fit data points. Select appropriate data set for your experiment and draw graphs.

Aim:
To implement the non-parametric Locally Weighted Regression (LWR) algorithm to fit a given set of data points and visualize the fitted curve using graphs.

Theory:
Locally Weighted Regression (LWR) is a non-parametric supervised learning algorithm used for regression problems. It uses distance-based weighting instead of global parameters.
Working Principle:
For a given query point x_0:

1. Compute distance from all training points
2. Assign weights using a kernel function
3. Fit a weighted linear regression
4. Predict output for x_0

Program:

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(0)
X = np.linspace(0, 2*np.pi, 100)
y = np.sin(X) + 0.1*np.random.randn(100)

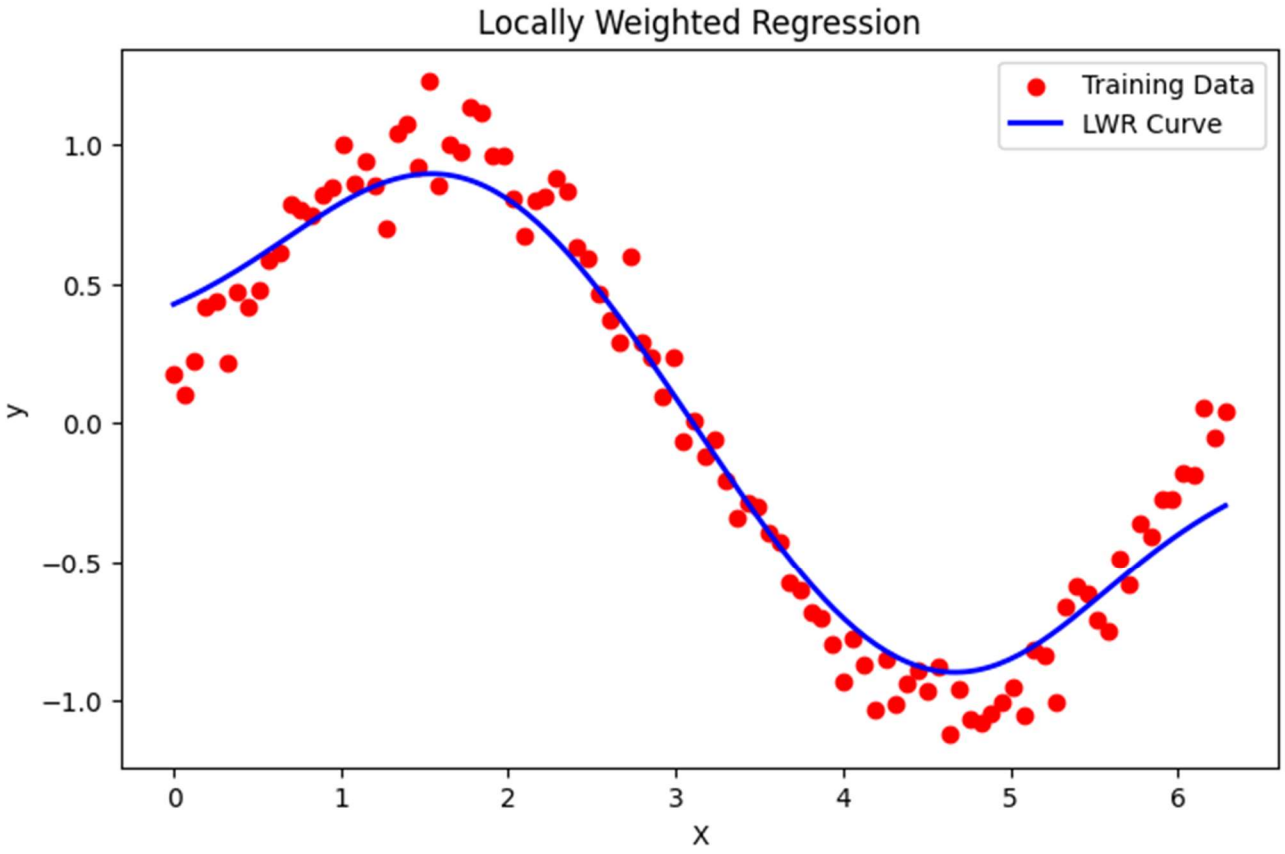
def weight(x, xi, tau):
    return np.exp(-(x - xi)**2 / (2 * tau**2))

def lwr(x0, X, y, tau):
    weights = np.array([weight(x0, xi, tau) for xi in X])
    return np.sum(weights * y) / np.sum(weights)

x_test = np.linspace(0, 2*np.pi, 200)
tau = 0.5
y_pred = np.array([lwr(x, X, y, tau) for x in x_test])

plt.figure(figsize=(8,5))
plt.scatter(X, y, color='red', label="Training Data")
plt.plot(x_test, y_pred, color='blue', label="LWR Curve", lin-
ewidth=2)
plt.xlabel("X")
plt.ylabel("y")
plt.title("Locally Weighted Regression")
plt.legend()
plt.show()
```

Output:


Locally Weighted Regression
Training Data
LWR Curve
1.0
0.5
0.0
−0.5
−1.0
y
0
1
2
3
4
5
6
X

Program 7. Linear Regression and Polynomial Regression

Develop a program to demonstrate the working of Linear Regression and Polynomial Regression. Use Boston Housing Dataset for Linear Regression and Auto MPG Dataset (for vehicle fuel efficiency prediction) for Polynomial Regression.

Aim:
To implement and demonstrate the working of Linear Regression using the Boston Housing Dataset and Polynomial Regression using the Auto MPG Dataset, and to analyze their performance in predicting continuous values.

Theory:
1. Linear Regression
Linear Regression is a **supervised learning algorithm** used to model the relationship between a **dependent variable** and one or more **independent variables** using a straight line. It assumes a **linear relationship** between input and output.
2. Polynomial Regression
Polynomial Regression is an extension of Linear Regression where the relationship between input and output is **non-linear**. It fits a **curved line** instead of a straight line.

Performance Metrics:
MSE evaluates prediction error, while **R^2** evaluates model quality. Both are essential for assessing regression performance.

1. **Mean Squared Error (MSE):**

Mean Squared Error measures the **average of squared differences** between actual and predicted values.

2. **R^2 Score:**

R^2 score measures **how much variance in the output is explained by the model**.

Program:

```
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures, Standard-
Scaler
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score

def linear_regression_boston():
    housing = pd.read_csv("Boston_housing_dataset.csv")  # Your
dataset file
    X = housing[["RM"]]  # Assuming 'RM' = average number of
rooms
    y = housing["MEDV"]  # Assuming 'MEDV' = median value of
homes

    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
```

```
    model = LinearRegression()
    model.fit(X_train, y_train)

    y_pred = model.predict(X_test)

    plt.scatter(X_test, y_test, color="blue", label="Actual")
    plt.plot(X_test, y_pred, color="red", label="Predicted")
    plt.xlabel("Average number of rooms (RM)")
    plt.ylabel("Median value of homes (MEDV)")
    plt.title("Linear Regression - Boston Housing Dataset")
    plt.legend()
    plt.show()

    print("Linear Regression - Boston Housing Dataset")
    print("Mean Squared Error:", mean_squared_error(y_test,
y_pred))
    print("R^2 Score:", r2_score(y_test, y_pred))

# Call the function
linear_regression_boston()


def polynomial_regression_auto_mpg():
    data = pd.read_csv('mpg.csv')

    X = data["displacement"].values.reshape(-1, 1)  # Reshape for
1 feature
    y = data["mpg"].values

    # Train-test split
    X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)

    # Polynomial features
    poly = PolynomialFeatures(degree=2)
    X_train_poly = poly.fit_transform(X_train)  # Transform
training data
    X_test_poly = poly.transform(X_test)        # Transform test
data

    scaler = StandardScaler()
    X_train_poly_scaled = scaler.fit_transform(X_train_poly)
    X_test_poly_scaled = scaler.transform(X_test_poly
    # Fit the linear regression model
    model = LinearRegression()
    model.fit(X_train_poly_scaled, y_train)

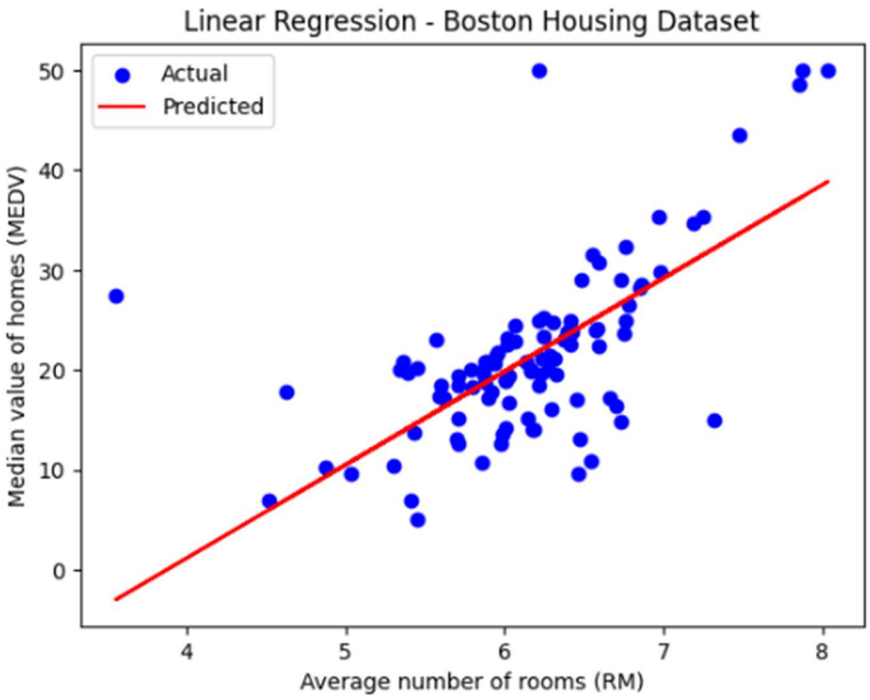
    # Predict
    y_pred = model.predict(X_test_poly_scaled)
```

```
    # Plot actual vs predicted
    plt.scatter(X_test, y_test, color="blue", label="Actual")
    plt.scatter(X_test, y_pred, color="red", label="Predicted")
    plt.xlabel("Displacement")
    plt.ylabel("Miles per gallon (mpg)")
    plt.title("Polynomial Regression - Auto MPG Dataset")
    plt.legend()
    plt.show()

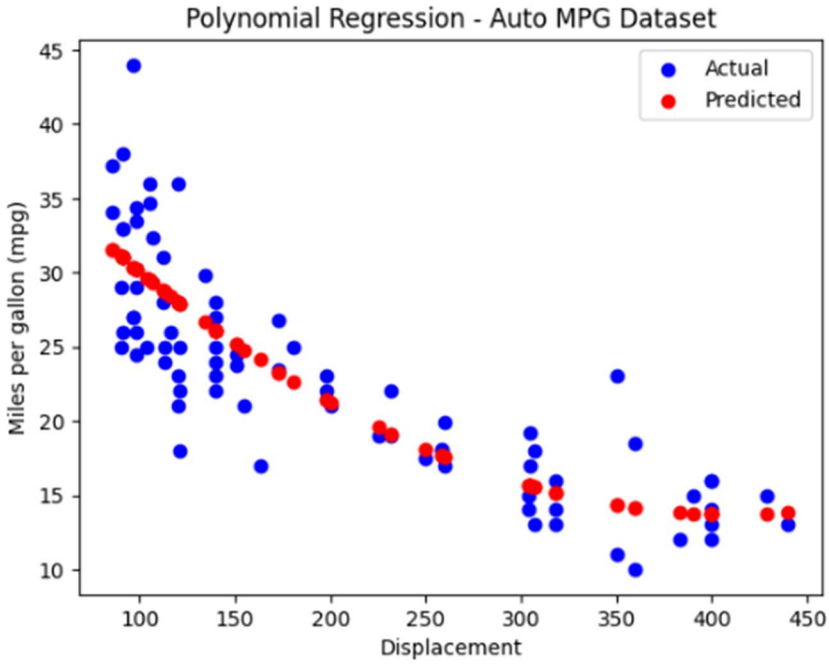
    # Print metrics
    print("Polynomial Regression - Auto MPG Dataset")
    print("Mean Squared Error:", mean_squared_error(y_test,
y_pred))
    print("R^2 Score:", r2_score(y_test, y_pred))

# Call the function
polynomial_regression_auto_mpg()
```

Output:




```
Linear Regression - Boston Housing Dataset
Mean Squared Error: 46.144775347317264
R^2 Score: 0.3707569232254778
```




```
Polynomial Regression - Auto MPG Dataset
Mean Squared Error: 15.107353561081197
R^2 Score: 0.7190189176110202
```

Program.8. Decision Tree Algorithm.

Develop a program to demonstrate the working of the decision tree algorithm. Use Breast Cancer Data set for building the decision tree and apply this knowledge to classify a new sample.

Aim:
To implement the Decision Tree algorithm using the Breast Cancer dataset and use the trained model to classify a new sample.

Theory:
Decision Tree
A Decision Tree is a supervised machine learning algorithm used for both:

- Classification
- Regression

It works by repeatedly splitting the dataset based on the best feature.
Breast Cancer Dataset
This dataset contains features computed from breast cell images.
Target:

- M→ Malignant (Cancer)
- B → Benign (No cancer)

Program:

```
# Importing necessary libraries
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn import tree

data = load_breast_cancer()
X = data.data
y = data.target

X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2, random_state=42)
clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:",accuracy * 100)
new_sample =X_test[:1]
prediction = clf.predict(new_sample)

prediction_class = "Benign" if prediction == 1 else "Malignant"
print("Predicted Class for the new sample:", prediction_class)
```

```
plt.figure(figsize=(12,8))
tree.plot_tree(clf, filled=True, feature_names=data.fea-
ture_names, class_names=data.target_names)
plt.title("Decision Tree - Breast Cancer Dataset")
plt.show()
```

Program (Using Local Dataset):

```
# Importing necessary libraries
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score
from sklearn import tree

# Load local dataset
data = pd.read_csv("BreastCancerDataset.csv")

# Drop ID column
data = data.drop(columns=['id'])

# Convert diagnosis labels to numbers
data['diagnosis'] = data['diagnosis'].map({'M': 0, 'B': 1})

X = data.drop('diagnosis', axis=1)
y = data['diagnosis']

X_train, X_test, y_train, y_test = train_test_split(
    X.values, y.values, test_size=0.2, random_state=42
)

clf = DecisionTreeClassifier(random_state=42)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:" ,accuracy * 100)

new_sample = np.array([X_test[0]])
prediction = clf.predict(new_sample)

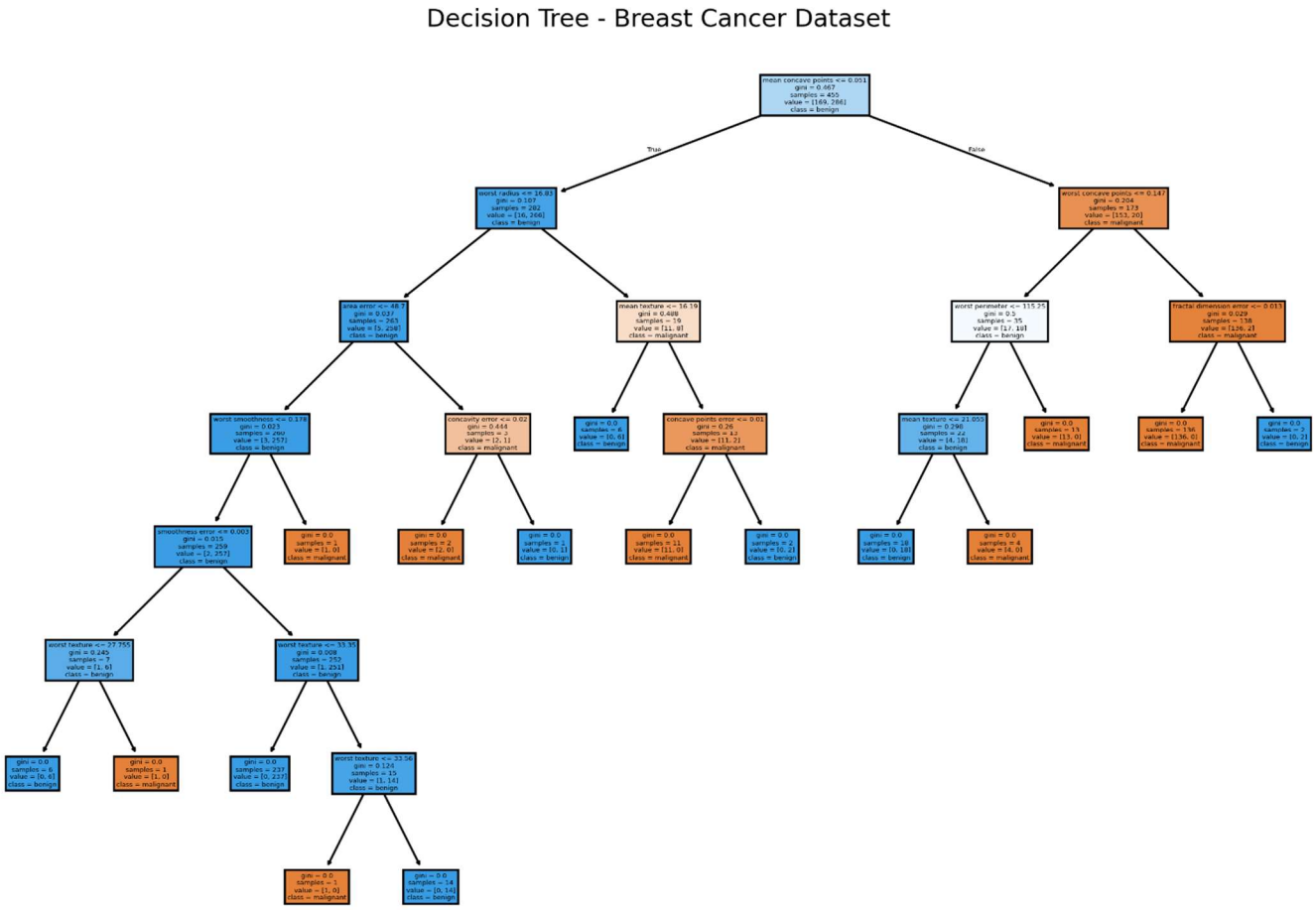
prediction_class = "Benign" if prediction == 1 else "Malignant"
print(f"Predicted Class for the new sample: {prediction_class}")

plt.figure(figsize=(12,8))
tree.plot_tree(
    clf,
    filled=True,
```

```
        feature_names=data.drop('diagnosis', axis=1).columns,
        class_names=["Malignant", "Benign"]
    )
    plt.title("Decision Tree - Breast Cancer Dataset")
    plt.show()
```

Output:

```
Model Accuracy: 94.73684210526315
Predicted Class for the new sample: Benign
```



Program 9: Naïve Bayes Classifier

Develop a program to implement the Naive Bayesian classifier considering Olivetti Face Data set for training. Compute the accuracy of the classifier, considering a few test data sets.

Aim:
To implement the Naive Bayes classifier using the Olivetti Face dataset and compute the accuracy of the model by classifying a few test images.

Theory:
Naive Bayes Classifier
Naive Bayes is a supervised probabilistic classifier based on Bayes' Theorem. It assumes that all features are independent of each other (naive assumption).

Program:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score

# Load data
images = np.load("olivetti_faces.npy")

# Prepare data
X = images.reshape((400, -1))    # 400 x 4096
y = np.repeat(np.arange(40), 10)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
model = GaussianNB()
model.fit(X_train, y_train)

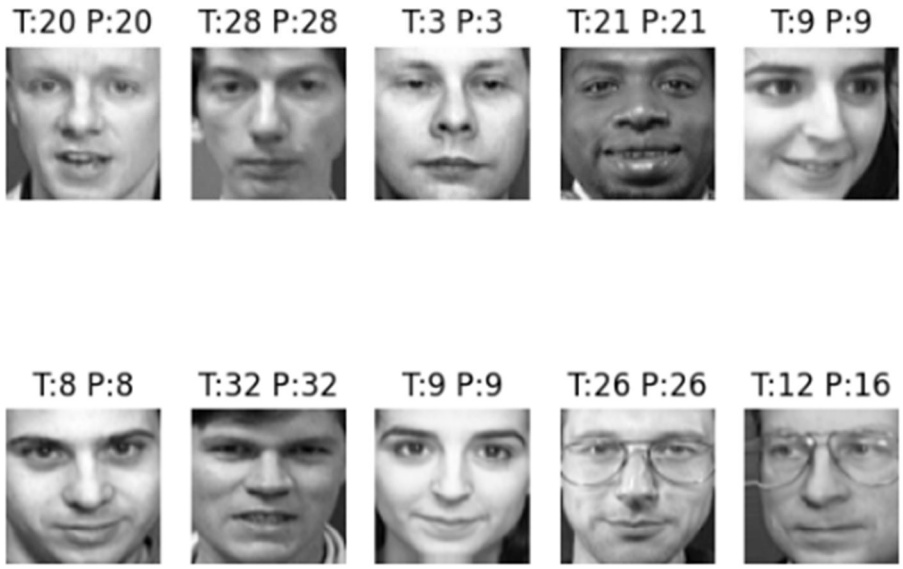
# Predict
y_pred = model.predict(X_test)

# Accuracy
print("Accuracy:", accuracy_score(y_test, y_pred) * 100)
# Show 10 test images
for i in range(10):
    plt.subplot(2, 5, i+1)
    plt.imshow(X_test[i].reshape(64,64), cmap='gray')
    plt.title(f"T:{y_test[i]} P:{y_pred[i]}")
    plt.axis('off')

plt.show()
```

Output:

```
Accuracy: 74.16666666666667
```



Program 10: K-Means clustering.
Develop a program to implement k-means clustering using Wisconsin Breast Cancer data set and visualize the clustering result.

Aim:
To implement the K-Means clustering algorithm on the Wisconsin Breast Cancer dataset and visualize the clustering results.

Theory:
K-Means is an unsupervised learning algorithm used to group data into K clusters based on similarity. It works without using class labels.

Working Principle

1. Choose number of clusters K
2. Randomly initialize K centroids
3. Assign each point to nearest centroid
4. Recalculate centroids
5. Repeat until convergence

Program:

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_breast_cancer
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

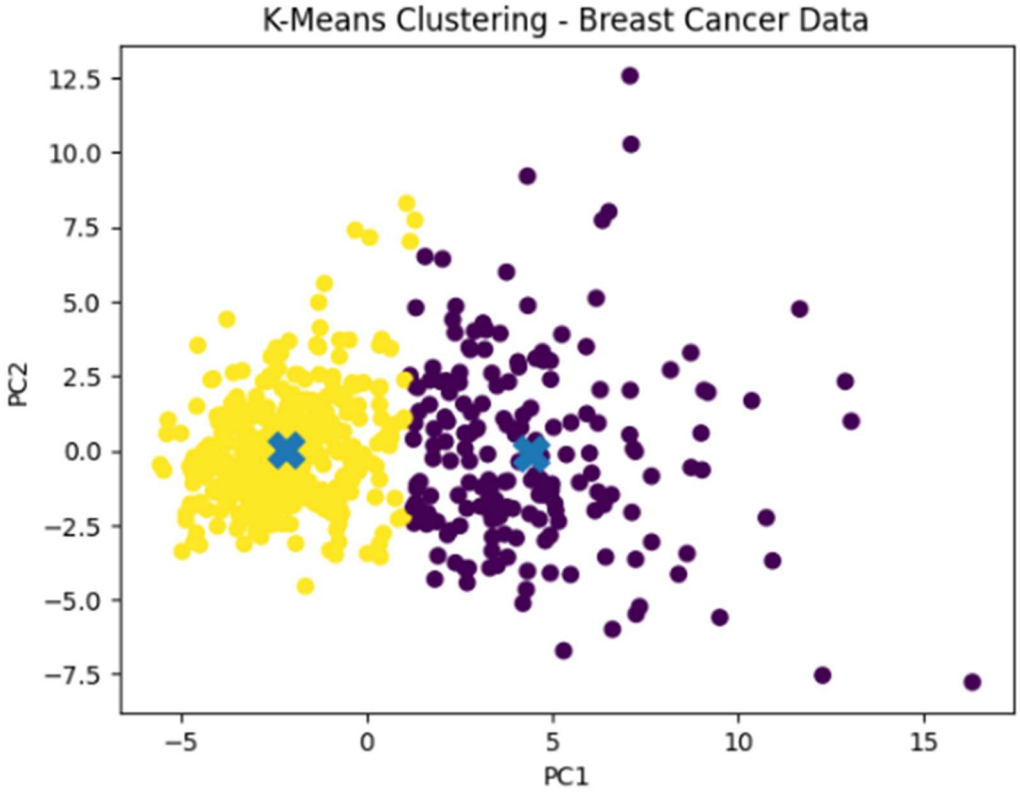
# Load data
data = load_breast_cancer()
X = data.data

# Scale data
X = StandardScaler().fit_transform(X)

# K-Means
kmeans = KMeans(n_clusters=2, random_state=42)
labels = kmeans.fit_predict(X)

# Reduce to 2D using PCA
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X)
centers = pca.transform(kmeans.cluster_centers_)
# Plot
plt.scatter(X_pca[:,0], X_pca[:,1], c=labels)
plt.scatter(centers[:,0], centers[:,1], marker='X', s=200)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("K-Means Clustering - Breast Cancer Data")
plt.show()
```

Output:


K-Means Clustering - Breast Cancer Data
12.5
10.0
7.5
5.0
2.5
0.0
−2.5
−5.0
−7.5
PC2
−5
0
5
10
15
PC1